Today task

1. Change batch size to (32,128 and 256)
2. Change number of epochs (20) max
3. Add one more layer (hidden) [RELU as activation]
4. Make more complex with (128 64 32) [as hidden layer]

In [1]:
```
import base_model
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mn
ist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarni
ng: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequent
ial models, prefer using an `Input(shape)` object as the first layer in the model in
stead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	7,850

```
Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9137 - loss: 0.2991
Test accuracy: 0.9254999756813049
```

In [2]:
```
# Change number of epochs to 20
Bm = base_model
Bm.model.fit(Bm.X_train, Bm.y_train, epochs=20, batch_size=64,verbose=0)
tloss, tacc = Bm.model.evaluate(Bm.X_train, Bm.y_train)
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9365 - loss: 0.2311
```

In [3]:
```
# Change batch size to (32,128 and 256)
batch_list = [32,64,128,256]
result = []
for bth in batch_list:
  his = Bm.model.fit(Bm.X_train, Bm.y_train, epochs=10, batch_size=bth,verbose=0)
  tloss, tacc = Bm.model.evaluate(Bm.X_train, Bm.y_train)
  result.append({
      'Batch size':bth,
      'Train loss': tloss,
      'Train accuracy': tacc
  })
print("Result of different batch size :- \n")
for i in result :
  print(i)
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9380 - loss: 0.2250
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9384 - loss: 0.2234
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9383 - loss: 0.2203
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9392 - loss: 0.2186
Result of different batch size :-

{'Batch size': 32, 'Train loss': 0.22668197751045227, 'Train accuracy': 0.9376000165
939331}
{'Batch size': 64, 'Train loss': 0.22426290810108185, 'Train accuracy': 0.9383000135
421753}
{'Batch size': 128, 'Train loss': 0.22147062420845032, 'Train accuracy': 0.938583314
4187927}
{'Batch size': 256, 'Train loss': 0.22010652720928192, 'Train accuracy': 0.939116656
7802429}
```

In [4]:

```python
# Task 3: Add one more layer (hidden) [RELU as activation]
model_2 = Bm.Sequential([
    Bm.Dense(96,activation="relu",input_shape=(784,)), # Add one Hidden layer
    Bm.Dense(10,activation="softmax")
])
model_2.summary()
model_2.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model_2.fit(Bm.X_train, Bm.y_train, epochs=10, batch_size=64,verbose=0)
tloss, tacc = model_2.evaluate(Bm.X_train, Bm.y_train)
print("Train loss :- ",tloss)
print("Train accuracy :- ",tacc)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 96)	75,360
dense_2 (Dense)	(None, 10)	970

Total params: 76,330 (298.16 KB)

Trainable params: 76,330 (298.16 KB)

Non-trainable params: 0 (0.00 B)

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9960 - loss: 0.0175
Train loss :-  0.01779070310294628
Train accuracy :-  0.9958833456039429
```

In [15]:

```python
# Task 4 : Make more complex with (128 64 32) [as hidden layer]
model_3 = Bm.Sequential([
    Bm.Dense(128,activation="relu",input_shape=(784,)), # Add one Hidden layer
    Bm.tf.keras.layers.BatchNormalization(),
    Bm.tf.keras.layers.Dropout(0.4),
    Bm.Dense(64,activation="relu"),
    Bm.tf.keras.layers.BatchNormalization(),
    Bm.tf.keras.layers.Dropout(0.2),
    Bm.Dense(32,activation="relu"),
    Bm.tf.keras.layers.BatchNormalization(),
```

```
    Bm.tf.keras.layers.Dropout(0.1),
    Bm.Dense(10,activation="softmax")
])
model_3.summary()
model_3.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model_3.fit(Bm.X_train, Bm.y_train, epochs=30, batch_size=32,verbose=0)
tloss, tacc = model_3.evaluate(Bm.X_train, Bm.y_train)
print("Train loss :- ",tloss)
print("Train accuracy :- ",tacc)
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 128)	100,480
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 64)	8,256
batch_normalization_4 (BatchNormalization)	(None, 64)	256
dropout_4 (Dropout)	(None, 64)	0
dense_14 (Dense)	(None, 32)	2,080
batch_normalization_5 (BatchNormalization)	(None, 32)	128
dropout_5 (Dropout)	(None, 32)	0
dense_15 (Dense)	(None, 10)	330

Total params: 112,042 (437.66 KB)

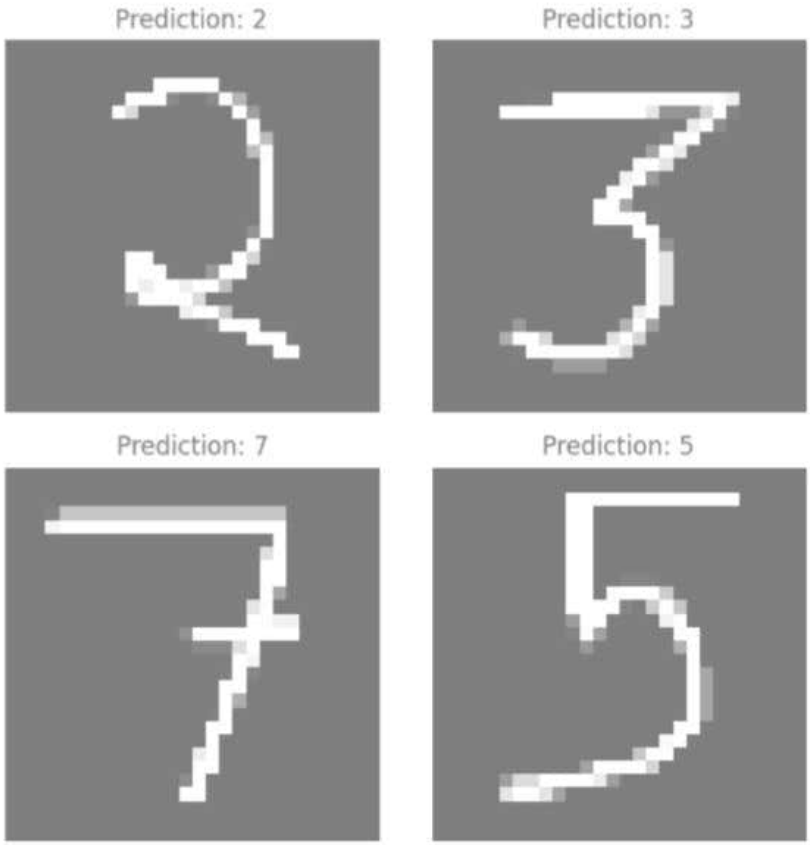
Trainable params: 111,594 (435.91 KB)

Non-trainable params: 448 (1.75 KB)

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9919 - loss: 0.0264
Train loss :-  0.02515314519405365
Train accuracy :-  0.9925000071525574
```

Now test on real data

In [18]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

files = []
```

```
for i in range (1,5):
  files.append(f"f{i}.png")
files

images = []
orig_images = []

for f in files:
    img = cv2.imread(f, 0)                  # grayscale
    img = cv2.resize(img, (28,28))
    orig_images.append(img)                 # display ke liye save
    img = img / 255.0                       # normalize
    img = 1 - img                           # invert pixels
    images.append(img)

images = np.array(images).reshape(-1,784)

pred = model_3.predict(images)
pred_class = np.argmax(pred, axis=1)

# show images with prediction
plt.figure(figsize=(6,6))

for i in range(len(orig_images)):
    plt.subplot(2, 2, i+1)                  # 2x2 grid
    plt.imshow(orig_images[i], cmap="gray")
    plt.title(f"Prediction: {pred_class[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```